Automated Method for Verification of Stochastic Graph Transformation Systems

Alavieh Sadat Alavizadeh
Islamic Azad University-Zarandiehbranch
Zarandieh, Iran
alavi_188@yahoo.com

Amirhossein Afradi
High Education Institute of Daneshestan
Saveh, Iran
amirafradi@yahoo.com

Abstract—**Stochastic Graph Transformation Systems (SGTS) have been introduced to model dynamic distributed systems. To insure correctness of a SGTS, we should employ verification concept. There are different methods for verifying such systems, for example simulation, test, deductive verification and model checking of which the most common is model checking. But most of the researches in the field of model checking come back to construct of a suitable model from SGTS and there is not done a complete research and a practical work in the field of model verification methods. Therefore, in this paper, we have implemented a SGTS to PRISM (a stochastic model checker) translator to automatically verify SGTS.**

Keywords-Stochastic graph transformation system; Verification; Model checking; Prism;

I. INTRODUCTION

To ensure that the system is really designed as intended, i.e. with no bugs, we need to test it in all possible environments it can appear in. Unfortunately, we cannotpractically perform such a complete test. The problem is that using test vectors we can only describe a few of all possible situations in which the system can appear[1].

Computer science brings us methods which offer a solution for the problem we have just sketched out. These methods are called *formal methods*. They can be viewed as a collection of mathematical methods that allow one to *verify* correctness of the model of the system in early design phases. The term *verification* means in this case taking into account all the possible environments the model can appear in[1].

Widest validation methods for hardware and software systems are *simulation*, *testing*, *deductive verification* and *model checking,* but for our work, model checking is the best method because *Model checking* is a technique for verifying finite state concurrent systems and verification can be performed *automatically*. The procedure normally uses an exhaustive search of the state space of the system to determine if some specification is true or not Given sufficient resources, the procedure always terminate with a **yes/no** answer[2].

Before model checking and verification, we should translate system model and its specifications to a model checker input language for example PRISM or SMART. In this paper we suggest an algorithm for translation of model and specifications to PRISM input language.

This paper is structured as follows. In section 2 we introduce stochastic graph transformation system and PRISM and in section 3 we introduce an algorithm for translation model and specifications to PRISM language.

II. BACKGROUND

A. Graph Transformation Systems

In this subsection, we describe graph transformation briefly, as a modeling means. For more information about theoretical background and semantics of graph transformation, interested readers can refer to [4,5,6].

Definition 1 (*attributed typed graph transformation*). An attributed typed graph transformation system is a triple *AGT=(TG,HG,R),* where *TG* is the type graph, *HG* is the host graph and *R* is the set of rules.

Definition 2 (T*ype Graph*). Let TG_Nbe a set of node types and TG_E be a set of edge types. Then a type graph *TG* is a tuple: $TG=(TG_N,TG_E,src,trg)$, with two functions $src: TG_E \rightarrow TG_N$and $trg: TG_E \rightarrow TG_N$ that assign to each edge a source and a target node. Each node type *NT* in TG_N is a triple: *NT=(Mult, Attr,O)*, where *Mult* is the multiplicity of the node and it is a pair: *Mult=(min,max)*, which $min \leq max$ and shows the minimum and maximum nodes of type *NT* in the host graphs. *Attr* is the set of its attributes. *O* is the set of its outgoing edges with corresponding multiplicity and destination node.

Definition 3 (*Host Graph*). A host graph *HG*, also called *instance graph* over *TG*, is a graph equipped with a graph morphism$type_G: HG \rightarrow TG$ that assigns a type to every node and edge in *HG*.

Definition 5 (*Graph Rules*). A graph transformation rule *P* over an attributed type graph *TG* is given by $P = (L \xleftarrow{l} K \xrightarrow{r} R, type, NAC)$, where L, K and R are three host graphs typed over TG. L is the left hand side (LHS) of the rule, K is the gluing graph and R is the right hand side (RHS) of the rule.

The application of a rule to a host graph HG can be performed by:

- Finding a matching of L in HG and checking the NAC.

978-1-4244-9698-3/11/$26.00 ©2011 IEEE

- Removing all corresponding graph elements in the host graph HG that are part of the L and are not part of the gluing graph K, resulting a new graph IG.
- Gluing or connecting the IG with an image of the R together by adding new graph elements e such that $e \subseteq R$ and $e \bigcap L = \Phi$, obtaining a new host graph HG'. HG' is the result of the rule application.

Example. We illustrate our concepts with the well-known problem of *mobile system: types and configurations* from [6]. Figure 1 on the left shows the type graph of our running example and on the right shows a sample host graph over the type graph.

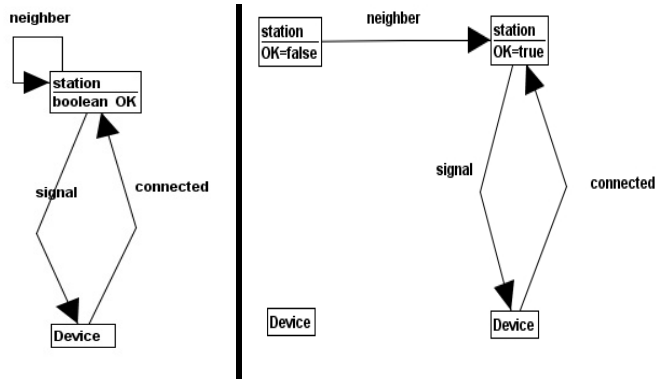
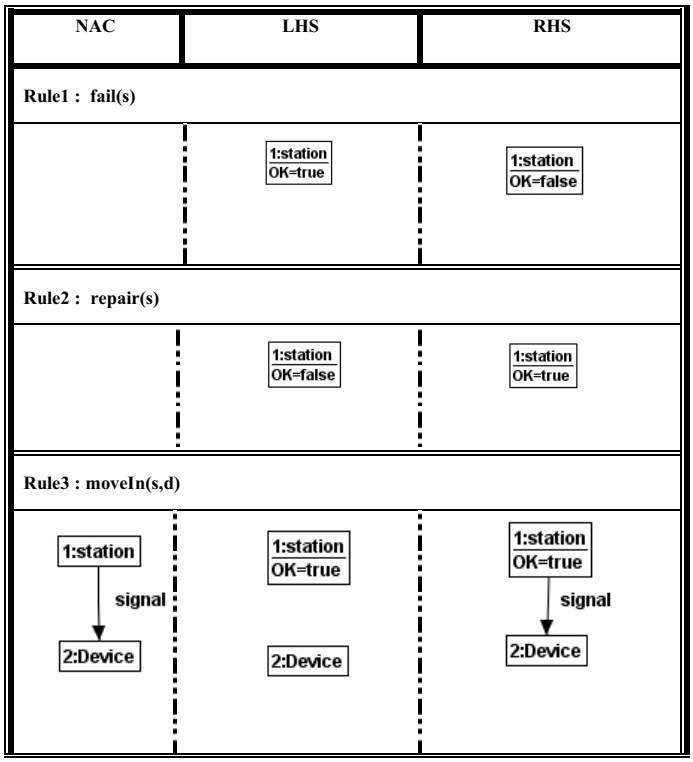
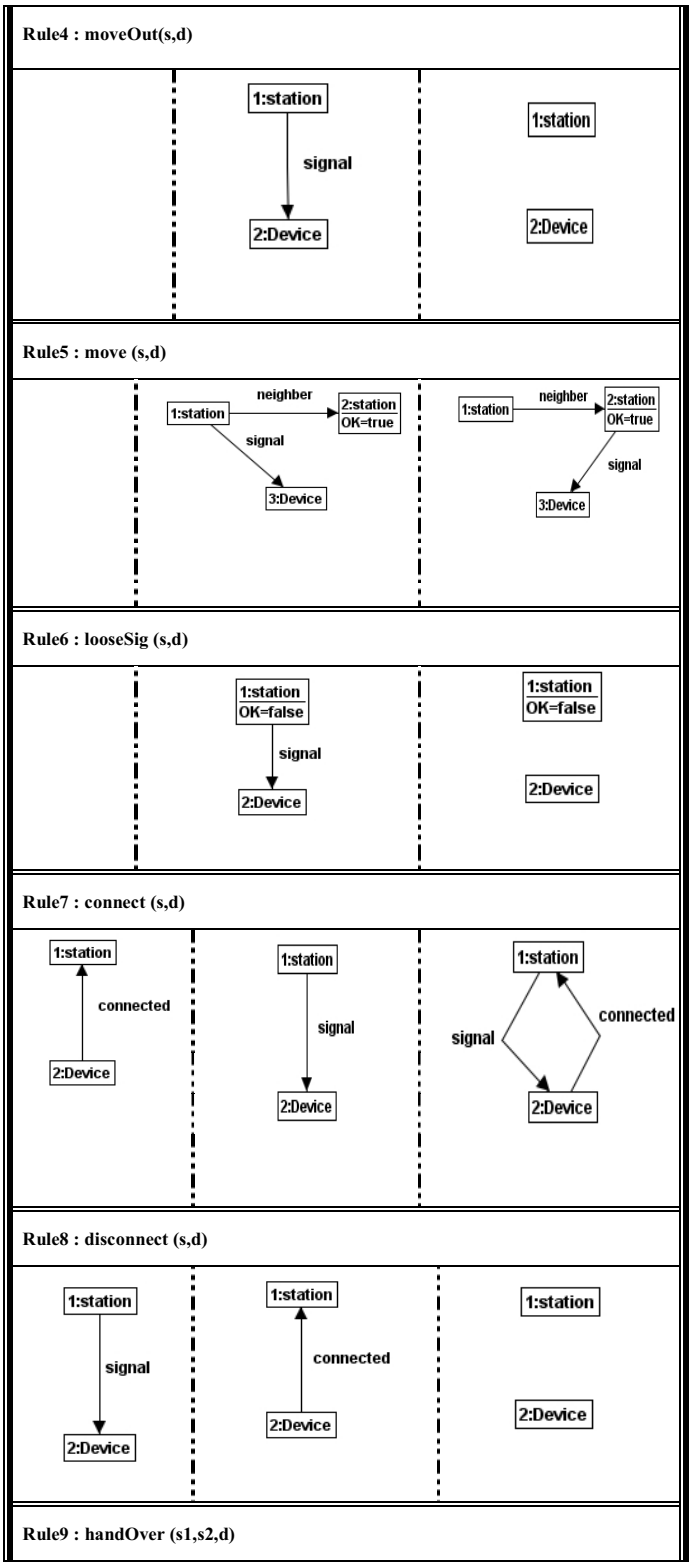


Figure 1.type graph and host graph

Graph transformation rules model the failure and recovery of components, their movement, the establishment and loss of connections, etc. We show rules in figure 2.





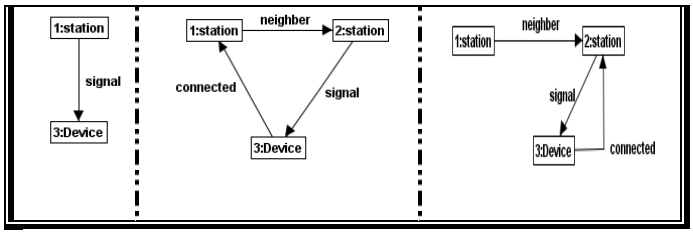


Figure 2.rules of stochastic graph transformation system

B. *Stochastic Graph Transformation*

In this subsectionwe review the basic conception of Stochastic Graph Transformationsystems [6].

Definition 6. (*Stochastic Graph Transformation Systems***)** A Stochastic Graph Transformation system consists of a Graph Transformation system and a function rate (p): P → R+ associating with every rule P its application rate.

The sample application rates for the rules of our example ofmobilesystem are shown in the Table 1:

Table1. Rate of rules of mobile system

Rule name p	*Rate(p)*	*Rule name p*	*Rate(p)*	*Rule name p*	*Rate(p)*
Repair	***0.1***	*Fail*	***0.2***	*Connect*	***0.7***
moveIn	***0.3***	*moveOut*	***0.4***	*Disconnect*	***0.8***
Move	***0.5***	*looseSig*	***0.6***	*Handover*	***0.9***

C. *The PRISM Model Checker*

In this subsectionwe introduce PRISM [3]. PRISM is a probabilistic model checker, a tool for the modeling and analysis of systems which exhibit probabilistic behavior. Probabilistic model checking is a formal verification technique. It is based on the construction of a precise mathematical model of a system which is to be analyzed. Properties of this system are then expressed formally in temporal logic and automatically analyzed against the constructed model.

PRISM basically supports three types of probabilistic models: Discrete-time Markov chains (DTMCs), Markov decision processes (MDPs), and Continuous-time Markov chains (CTMCs).To indicate which model type is being described; a PRISM model includes one of the keywords dtmc, mdp or ctmc.

The fundamental components of the PRISM language are modules and variables.

Local variables are declared in the standard way and for global declaration we should use global keyword.For example the following declaration declares a Boolean variable F1, initialized to false.

```
F1: bool init false;
```

While the following declaration, declares an integer P1 from 0 to 3 with 0 as initial state.

```
P1: [0..3] init 0;
```

The dynamic behavior of each module is described by a set of commands. A command takes the form:

```
[]  <guard>  -><rate_1> : <update_1> + ... + <rate_n> : <update_n>;
```

The guard is a predicate over all the variables in the model. Each update describes a transition which the module can make if the guard is true. A transition is specified by giving the new values of the variables in the module, each update is also assigned a rate which will be assigned to the corresponding transition.Rate is a positive real number which is used in Continuous-time Markov chains.

Once the file has been parsed successfully, the model can be built. If there are no errors during model construction, the number of states and transitions in the model will be displayed in the bottom left corner of the PRISM GUI window.

The presence of deadlock states in the model, i.e. states which are reachable but from which there are no outgoing transitions, constitutes an error.

1) *Property Specification*

In order to analyse a probabilistic model which has been specified and constructed in PRISM, it is necessary to identify one or more properties of the model which can be evaluated by the tool. In PRISM this is done using temporal logic.

The basic syntax of the language for expressing a PRISM property prop is given by the following grammar:

```
Prop ::= true | false | expr| !prop | prop& prop |
prop|prop |  prop => prop | P bound [pathprop] | S
bound [prop]
Bound ::= >=p | >p | <=p | <p
Pathprop ::= X prop | prop U prop | prop U time prop
| F prop |  F time prop | G prop | G time prop
Time ::=  >=t | <=t | [t,t]
```

Figure3. Basic syntax of the language for expressing a PRISM property prop

III. OUR PROPOSED APPROACH

We present our approach in 4 subsections. In first subsection we say that how we can translate stochastic graph transformation system into the PRISM input language. In second subsection we present specification verification of SGTS in PRISM. In third subsection, architecture of our algorithm is shown in a figure and finally in fourth subsection we describe our algorithm with a case study.

A. *algorithm for transforming stochastic graph transformation into the input of prism language*

Our algorithm comprises of these three following parts: Deciding on the type of model; Deciding on the necessary variables of this model; and translating the rules of the model. We will explain each one in details. We implemented this algorithm and entered SGTS that we drew in AGG software as input and got PRISM code as output.

*1) Deciding on the type of the model:*Since stochastic graph transformation systems are derived from continuous time Markov chains, we choose CTMC as the type of the model.

2) Deciding on necessary variables of the model: We can put these variables into two major categories: variables that are derived from attributes of each node and variables that appear due to the existence of an edge among nodes of graph.

a) Recognizing variables derived from the node's attributes

To recognize the variables that are derived from the attributes of nodes, we should work as follow:

a. Analyze all nodes of type graph

b. If a node has an attribute or attributes, for every attribute in type graph node, check whole of host graph nodes and describe variables to the number of host graph nodes that are of the same type with the mentioned node in type graph.

c. The name of each variable comprises of three parts:

Name of the node in type graph + Number of that particular node in host graph + name of the attribute in type graph

d. The type of variables equal to the recognized type of each attribute in type graph.

If the type of attribute is Boolean, consider the type of variable as Bool.

Else if the type of attribute is integer, consider an interval of all amounts that the variable can accept; To fulfill this task, analyze all values of this attribute in host graph and rules, estimate the maximum and minimum amounts and make interval related to this variable.

e. Assigning initial value to variables.

Refer to the host graph and analyze all host graph nodes that is of the same type with that particular node in type graph and consider the value of this attribute in each proper node as the initial value of the variable.

Figure4. Variables that are derived from the attributes of nodes

b) Recognizing variables that are derived from the edges among type graph nodes:

Analyze all existent edges in the type graph

Determine the name of source and target nodes

Number of variables for each edge = number of the source node samples in host graph * number of target node samples in host graph.

For definition of these variables, follow these steps:

1. Choosing the name of the variable:

Consider all nodes of host graph which are of the same type with source node

For each node of the same type with source node, consider all nodes of host graph that are of the same type with target node.

The name of each variable comprises five parts:

1. The name of source node
2. The number of node which is of the same type with source node in host graph
3. The name of the edge
4. The name of the target node
5. The number of nodes which are of the same type with the target node in host graph

2. Assigning the initial value of each variable:

Analyze all edges of host graph, for each node of the same type with target node of host graph.

If there is an edge between these two nodes of host graph under the study, the initial value of variable is true. Else it is false.

Figure5. Variables that are derived from the edges among type graph nodes

3) Translation of the rules:

Since each rule comprised of three parts: precondition or LHS, post condition or RHS and NAC, so for transforming each rule we go through these steps:

1. Consider an array for keeping all node samples that are used in the current rule so far.
2. Consider four factors for each node sample of host graph:
   - The number of node samples in the rule
   - Reputation or not reputation of the node
   - Whether the node was analyzed before or not
   - If the node was analyzed before, the number of states of that situation.
3. For each node of left side graph of rule go through these steps:

a. If the current node existed in the defined array in first step, it means that it was analyzed before and don't increase count of node samples in rule.

Else, add this node to the array and increase one unit to the number of node samples in rule.

b. Analyze all attributes of node to find the name of each attribute and its amount and save them into one array.

c. Analyze all the edges of left side graph to find that whether there are some edges that this node is their sources:

If there are such edges, find the name of this edge and target node name and make some strings, equal to the count of target node samples in host graph, which each string is comprised of these three parts:

"The name of the edge + the name of the target node + sequential number from one to the number of samples of the target node"

If the target node wasn't analyzed before, consider it as an analyzed node and delete the edge.

d. Do the previous step to find out that if this node is the edge target or not. All these steps are like previous step, except: instead of the target node, use source node and vice versa.

e. Analyze all nodes in RHS and NAC graphs to know that whether this node is copied in these graphs or not.

If there are similar nodes, follow the previous steps for these graphs.

f. Follow next steps that show how choose from saved states:

Achieving the number of different states of the rule by including the present node:

Consider the number of states as one.

Analyze each type of the nodes in host graph, to find out the number of node samples in current rule, which were analyzed sofar.

If the number of the analyzed node samples equals one, we should multiply the number of states of rule by the number of the node samples in host graph

If the number of analyzed node samples is more than one, use next formula and multiply the result by the number of states of rule.

$$\frac{\text{(The number of node samples in host graph)!}}{\text{(The number of node samples in host graph – the number of checked node samples in current rule)!}}$$

For all nodes in host graph, check that how many samples of this node is checked in current rule so far.

If the number of analyzed node samples equals 1 then

Check that whether this sample of node is analyzed before in LHS, RHS and NAC graphs or not.

If this sample of node was found in one of the graphs, numerate for $\frac{the\ number\ \ of\ the\ rule\ \ states}{the\ number\ \ of\ \ node\ \ sampl\ es\ in\ host\ \ grap\ h}$ times that

this numbers are in $[1 \,..\, the\ number\ of\ node\ samples\ in\ host\ graph]$

If the number of analyzed node samples is more than one then

$$flo \leftarrow \frac{the\ number\ of\ states\ of\ current\ rule}{the\ number\ of\ node\ samples\ in\ host\ graph}$$

$$num \leftarrow the\ number\ of\ node\ samples\ in\ host\ graph$$

Assume one matrix

While flo>1 do

Set assignment index as 0

Do next steps for $\frac{the\ number\ of\ states\ of\ current\ rule}{flo}$ times

Set the assignment index value equal to the remainder of division of the assignment index to the number of node samples in host graph and increase this index one unit.

Check values of columns in top of current column. If value of one of those columns equals to assignment index go to previous step.

For flo times set the value of sequential items equal to assignment index.

Go to next row.

Decrease one unit of num.

If its value equals 1 then don't continue else $flo \leftarrow \frac{flo}{num}$

Check LHS, RHS and NAC graphs to determine that what sample of this node is there and depend on that sample, select one row of matrix.

g. Add selections of previous step to the values those are constructed before. For this, check all node types in host graph to know type of selected node from LHS and whether this node is checked for first time or not?

If this node is repeated then

If one sample of that is checked for first time then consider its value in first time for now.

Else consider the value as 1.

Else divide the number of rule states to the number of node samples in host graph and consider the result.

h. Depend on the selections of previous step, select one of the constructed strings and produce different rule states with the name of the selected node from LHS graph, the number of node samples in host graph and the number of valid iterations that made from current step.

4. If there is a node or edge in RHS and NAC graph that don't scanned before, go to step3

5. Produce the complete rule with whole rule states to PRISM language

Figure 6. Translation of rules

B. Verification of properties in PRISM

Study has showed that program verification and analysis is one of the most important processes in the life cycle of software. Informally, verification is proving the correctness of a program. Formally, verification is theoretical methods for proving the correctness of an algorithm, or program.

In 1977, Lamport observed that there are two fundamental classes of trace properties, namely *safety* and *liveness* properties. Briefly spoken, safety properties describe what is not allowed to happen. Liveness properties demand what eventually must happen. Every dependable property could be written as the intersection of a safety and a liveness property.

Based on the previous paragraph, we describe these two properties for verification of stochastic graph transformation system. For usage of PRISM for properties checking we should do following steps:

1. Definition of CSL[1] formulas
2. Consider properties as graph transformation rules that negative states are shown with NAC graph and positive states are shown with LHS graph and RHS graph equals to LHS graph.

1) Safety property

Definition of safety property is as follow:

$$\Box(\neg p)$$

We should produce P from properties that are shown as graph transformation system. For this purpose we can use from previous section that translates the rules but in this section we don't consider RHS graph and only translate LHS and NAC graphs and put OR operator between different states of current rule. Now we translate CSL formula to PRISM input language as next formula. If the result of verification is true, the safety property is supported in all states.

$$P >= 0.99\ [\ true\ \ U\ \ !("p")\ ]$$

If we translate CSL formula similar to next formula, if the result of verification is 1, the safety property is supported in all states.

$$P =?\ [\ true\ \ U\ \ !("p")\ ]$$

2) Liveness property

Definition of liveness property is as follow:

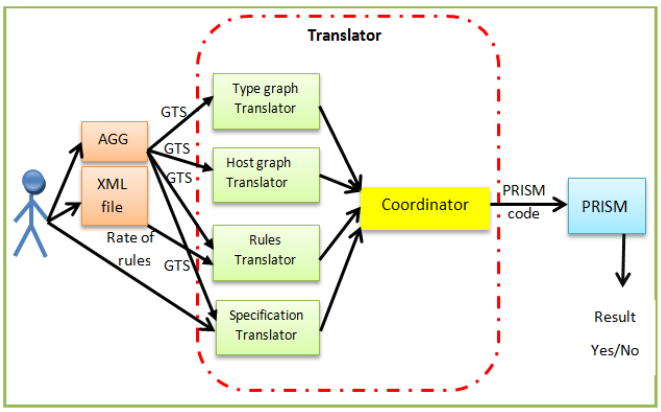
$$\Box(p1 \longrightarrow \Diamond\ p2)$$

Steps are similar to previous property but we there are two graph transformation rules as P1 and P2. Now we translate CSL formula to PRISM input language as next formula. If the result of verification is true, the Liveness property is supported in all states.

$$"p1" \Longrightarrow P >= 0.99\ [true\ \ U\ \ "p2"\ ]$$

C. Architecture of the Translator

In this section we show the steps of work completely in figure 7.



[1] Continuous Stochastic Logic

D. Case study

In this section we consider mobile network as a stochastic graph transformation system and use our proposed approach to this example.

Depend on figures 4, 5 and 8,we show the result of variable definitions to PRISM input language.

```
stat1OK : bool init false;
stat2OK : bool init true;

Devi1connstat1 : bool init false;
Devi1connstat2 : bool init false;
Devi2connstat1 : bool init false;
Devi2connstat2 : bool init true;
stat1signDevi1 : bool init false;
stat1signDevi2 : bool init false;
stat2signDevi1 : bool init false;
stat2signDevi2 : bool init true;
stat1neigstat1 : bool init false;
stat1neigstat2 : bool init true;
stat2neigstat1 : bool init false;
stat2neigstat2 : bool init false;
```

Figure 8.Defined variables

Figure 9 shows the result of rule translations to PRISM input language for some rules of our example.

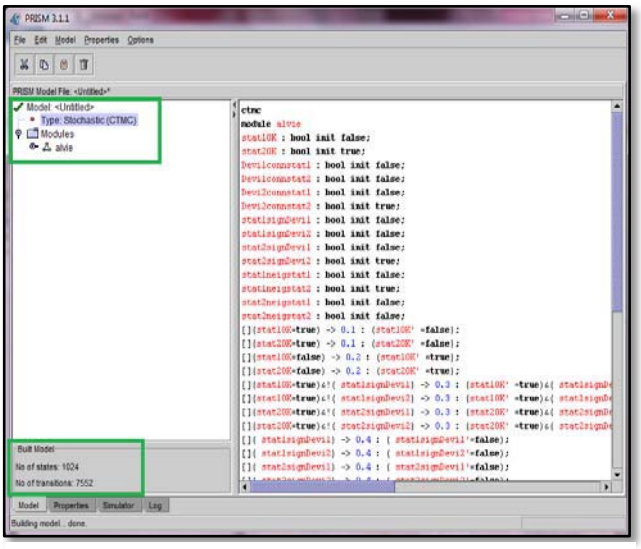
```
[](stat1OK=true) -> 0.1 : (stat1OK' =false);
[](stat2OK=true) -> 0.1 : (stat2OK' =false);

[](stat1OK=true)&!(stat1signDevi1) -> 0.3 : (stat1OK' =true)
&( stat1signDevi1'=true);
[](stat1OK=true)&!(stat1signDevi2) -> 0.3 : (stat1OK' =true)
&( stat1signDevi2'=true);
[](stat2OK=true)&!(stat2signDevi1) -> 0.3 : (stat2OK' =true)
&( stat2signDevi1'=true);
[](stat2OK=true)&!(stat2signDevi2) -> 0.3 : (stat2OK' =true)
&( stat2signDevi2'=true);

[]( stat1signDevi1)&(stat1OK=false) -> 0.6 : (stat1OK' =false)
&( stat1signDevi1'=false);
[]( stat1signDevi2)&(stat1OK=false) -> 0.6 : (stat1OK' =false)
&( stat1signDevi2'=false);
[]( stat2signDevi1)&(stat2OK=false) -> 0.6 : (stat2OK' =false)
&( stat2signDevi1'=false);
[]( stat2signDevi2)&(stat2OK=false) -> 0.6 : (stat2OK' =false)
&( stat2signDevi2'=false);

[](stat1signDevi1)& !(Devi1connstat1) -> 0.7 :
(stat1signDevi1'=true)&(Devi1connstat1'=true);
[](stat1signDevi2)& !(Devi2connstat1) -> 0.7 :
(stat1signDevi2'=true)&(Devi2connstat1'=true);
[](stat2signDevi1)& !(Devi1connstat2) -> 0.7 :
(stat2signDevi1'=true)&(Devi1connstat2'=true);
[](stat2signDevi2)& !(Devi2connstat2) -> 0.7 :
(stat2signDevi2'=true)&(Devi2connstat2'=true);
```

Figure 9. Translation of rules to the PRISM language

Figure 10shows that the constructed code from this SGTS is parsed and there isn't any error.

Safety property

As an example of safety property in mobile network system, see figure 11. According to this property, one device cannot connect to two stations in any time.



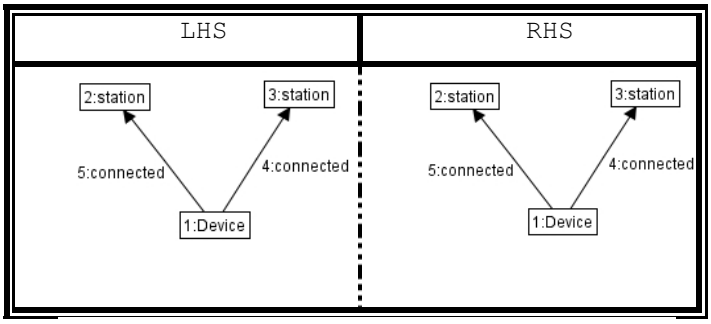


Figure11. Safety property with SGTS

Figure 12 shows translation of figure11 to the Prism input language in template of one label:

```
Label "safety" = ((( Devi1connstat2=true)&( Devi1connstat1=true)) |
((Devi2connstat2=true)&( Devi2connstat1=true)))
```

Figure12. Translation of figure 11 in template of a label

Now we put the label in following formula and verify the property:

$$P >= 0.99\ [\ true\ \ U\ \ !("safety")\ ]$$

The result of verification is shown in figure13.

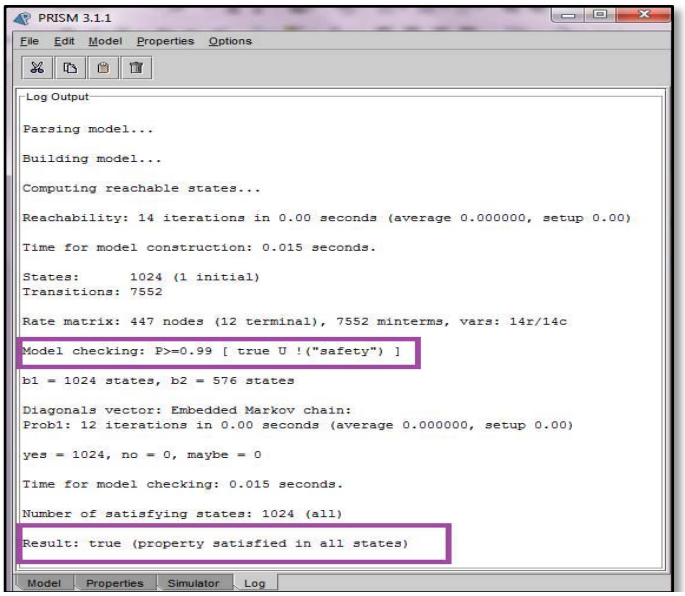


Figure13. Result of verification

REFERENCES

[1] J.Barnat, T. Brazdil, P. Krcal, V. Rehak, and D. Safranek, "Model checking in IPv6 Hardware Router Design", CESNET technical report 8, 2002.

[2] D. D'Aprile, "Timed and Stochastic Model Checking of Petri Nets", Doctoral Dissertation, Dipartimento di Informatica, UNIVERSIT`A DEGLI STUDI DI TORINO, Torino(Italia), 2007.

[3] PRISM website. Available: http://www.prismmodelchecker.org

[4]L. Baresi, and R. Heckel, "Tutorial Introduction to Graph Transformation: A Software Engineering Perspective", In Proc. of the First International Conference on Graph Transformation (ICGT), LNCS, vol. 2505, 402–429, 2002

[5] H.Ehrig, G.Engels, H.Kreowski, and G. Rozenberg (eds.): "Handbook on Graph Grammars and Computing by Graph Transformation". vol. 2: Applications, Languages and Tools. World Scientific, 1999

[6] R.Heckel, G.Lajios, and S.Menge, "Stochastic graph transformation systems", in H. ehrig, G. Engels, and F. Parisi-Presicce Eds. proc. Graph Transformations: Second International Conference, ICGT 2004, Rome, Italy,LNCS, vol.3256, pp. 210-225, Springer, ISBN 3-540-23207-9, 2004.

[7] N.A.Cuong, "A Study on Formal Program Verification through Program Properties : Liveness, Safety and Secutiry", technical report, School of Computing, National University of Singapore, Singapore, 2007.

[8] Z.Benenson, F.C.Freiling, T. Holz, D. Kesdogan, and D. Draque Penso, "Safety, Liveness, and Information Flow: Dependability Revisited", ARCS 2006 Workshops, pp. 56-65, 2006.